Feature Extraction and Feature Analysis

Lab exercises covering basic features, SIFT, ORB, and feature analysis, using Python, OpenCV, scikit-image and scikit-learn.

Basic Features

1. Pixel, Intensity and Histogram Features

Objective: Understand raw pixel values and global intensity statistics as the simplest features.

Tasks:

- Load an image, convert to grayscale, and resize to a fixed size.
- Flatten the pixels into a feature vector and report its dimension.
- Compute the mean, standard deviation, skewness and entropy of intensities.
- Plot and compare grayscale histograms (32 and 256 bins) for different images.
- Apply histogram equalization and compare before and after.

Sample code:

```
import cv2, numpy as np
from scipy.stats import skew, entropy
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
small = cv2.resize(gray, (64, 64))
raw_feature = small.flatten()
print('Raw feature dimension:', raw_feature.shape)

hist = cv2.calcHist([gray], [0], None, [256], [0, 256]).ravel()
hist_norm = hist / hist.sum()
stats = [gray.mean(), gray.std(), skew(gray.ravel()), entropy(hist_norm + 1e-12)]
print('Mean, Std, Skewness, Entropy:', stats)

eq = cv2.equalizeHist(gray)
plt.subplot(121); plt.hist(gray.ravel(), 256); plt.title('Original')
plt.subplot(122); plt.hist(eq.ravel(), 256); plt.title('Equalized')
plt.show()
```

Analysis questions:

- Why are raw pixels a poor feature under shifts, rotation, or lighting change?
- Which statistic best separates a dark image from a bright one?

2. Color Features

Objective: Extract and compare color descriptors across color spaces.

Tasks:

- Compute 3D color histograms in RGB and HSV.
- Compute color moments (mean, std, skewness per channel) to get a 9-value descriptor.
- Compare images using histogram distances (correlation, chi-square, intersection, Bhattacharyya).
- Test robustness by changing the brightness of an image.

Sample code:

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
h = cv2.calcHist([hsv], [0, 1, 2], None, [8, 8, 8], [0, 180, 0, 256, 0, 256])
h = cv2.normalize(h, h).flatten()

def color_moments(image):
   feats = []
   for c in cv2.split(image):
      c = c.astype(float).ravel()
      feats += [c.mean(), c.std(), skew(c)]
   return np.array(feats)

print(color_moments(hsv))

h1 = cv2.calcHist([img], [0], None, [64], [0, 256])
h2 = cv2.calcHist([cv2.convertScaleAbs(img, alpha=1, beta=50)], [0], None, [64], [0, 256])
for name, m in [('Correlation', cv2.HISTCMP_CORREL), ('Chi-square', cv2.HISTCMP_CHISQR),
            ('Intersection', cv2.HISTCMP_INTERSECT), ('Bhattacharyya', cv2.HISTCMP_BHATTACHARYYA)]:
   print(name, cv2.compareHist(h1, h2, m))
```

Analysis questions:

- Which color space is more stable under illumination change, RGB or HSV?
- How does the number of bins affect descriptor size and discriminative power?

3: Edge and Gradient Features

Objective: Use gradients and edges as features and study their sensitivity to noise.

Tasks:

- Apply Sobel, Prewitt, Laplacian and Canny to the same image.
- Add Gaussian noise and repeat. Compare results with and without Gaussian smoothing.
- Compute gradient magnitude and orientation, and build an orientation histogram.
- Vary the Canny thresholds and sigma, and tabulate the edge pixel counts.

Sample code:

```
blur = cv2.GaussianBlur(gray, (5, 5), 1.4)
gx = cv2.Sobel(blur, cv2.CV_64F, 1, 0, ksize=3)
gy = cv2.Sobel(blur, cv2.CV_64F, 0, 1, ksize=3)
mag = np.sqrt(gx**2 + gy**2)
```

```
ang = np.degrees(np.arctan2(gy, gx)) % 180

lap = cv2.Laplacian(blur, cv2.CV_64F)
canny = cv2.Canny(blur, 100, 200)

orient_hist, _ = np.histogram(ang, bins=9, range=(0, 180), weights=mag)
print('Edge orientation histogram:', orient_hist / orient_hist.sum())
```

Analysis questions:

- Why does the Laplacian amplify noise more than Sobel?
- How would you choose Canny thresholds automatically (for example from the median intensity)?

Shape Features

Objective: Describe objects by their contours and shape moments.

Tasks:

- Segment objects using Otsu thresholding and find contours.
- Compute area, perimeter, aspect ratio, extent, solidity and circularity for each contour.
- Compute Hu moments (7 values) and test their invariance to rotation and scale.
- Match shapes with cv2.matchShapes.

Sample code:

```
_, bw = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
contours, _ = cv2.findContours(bw, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

for c in contours:
    area = cv2.contourArea(c)
    if area < 100: continue
    perim = cv2.arcLength(c, True)
    x, y, w, h = cv2.boundingRect(c)
    hull_area = cv2.contourArea(cv2.convexHull(c))
    circularity = 4 * np.pi * area / (perim ** 2)
    solidity = area / hull_area
    hu = cv2.HuMoments(cv2.moments(c)).flatten()
    hu_log = -np.sign(hu) * np.log10(np.abs(hu) + 1e-30)
    print(area, perim, w / h, circularity, solidity, hu_log)
```

Analysis questions:

- Which shape features are scale invariant? Rotation invariant?
- Why are Hu moments usually shown on a log scale?

5: Texture Features (LBP and GLCM)

Objective: Quantify texture using statistical and local pattern methods.

Tasks:

- Compute LBP histograms with different radius and number of points.
- Build GLCMs at several distances and angles, and compute contrast, correlation, energy and homogeneity.
- Compare smooth, rough and regular textures (for example Brodatz or KTH-TIPS).

Sample code:

```
from skimage.feature import local_binary_pattern, graycomatrix, graycoprops

P, R = 8, 1
lbp = local_binary_pattern(gray, P, R, method='uniform')
lbp_hist, _ = np.histogram(lbp, bins=P + 2, range=(0, P + 2), density=True)

glcm = graycomatrix(gray, distances=[1, 3], angles=[0, np.pi/4, np.pi/2, 3*np.pi/4],
                    levels=256, symmetric=True, normed=True)
for prop in ['contrast', 'correlation', 'energy', 'homogeneity']:
    print(prop, graycoprops(glcm, prop).mean())
```

Analysis questions:

- What does the 'uniform' LBP mode do, and why does it shorten the histogram?
- Which GLCM property separates smooth from rough texture best?

6: Corner Detection

Objective: Detect interest points and measure their stability.

Tasks:

- Apply Harris, Shi-Tomasi (goodFeaturesToTrack) and FAST.
- Test on rotated, scaled and blurred versions of the same image.
- Compute a simple repeatability score: the fraction of keypoints found again after the transformation.

Sample code:

```
harris = cv2.cornerHarris(np.float32(gray), blockSize=2, ksize=3, k=0.04)
harris_pts = np.argwhere(harris > 0.01 * harris.max())

shi = cv2.goodFeaturesToTrack(gray, maxCorners=200, qualityLevel=0.01, minDistance=10)

fast = cv2.FastFeatureDetector_create(threshold=30)
fast_kp = fast.detect(gray, None)
print(len(harris_pts), len(shi), len(fast_kp))
```

Analysis questions:

- Which detector is fastest, and which is most repeatable under rotation and scale?
- Why is Harris rotation invariant but not scale invariant?

Local Feature Descriptors

7: SIFT (Scale-Invariant Feature Transform)

Objective: Detect and describe scale- and rotation-invariant keypoints, and match them between images.

Tasks:

- Create a SIFT detector with cv2.SIFT_create() and extract keypoints and 128-dimensional descriptors.
- Draw keypoints with DRAW_RICH_KEYPOINTS to see scale and orientation.
- Match two images with a brute-force matcher and with FLANN, then apply Lowe's ratio test (0.75).
- Estimate a homography with RANSAC and locate the object in a scene.
- Vary nfeatures, contrastThreshold and sigma, and record the effect on keypoint count and matching quality.

Sample code:

```
img1 = cv2.imread('object.jpg', cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread('scene.jpg', cv2.IMREAD_GRAYSCALE)

sift = cv2.SIFT_create(nfeatures=0, contrastThreshold=0.04, edgeThreshold=10, sigma=1.6)
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)
print('Keypoints:', len(kp1), len(kp2), 'Descriptor shape:', des1.shape)

# FLANN matcher (KD-tree) with ratio test
flann = cv2.FlannBasedMatcher(dict(algorithm=1, trees=5), dict(checks=50))
knn = flann.knnMatch(des1, des2, k=2)
good = [m for m, n in knn if m.distance < 0.75 * n.distance]
print('Good matches:', len(good))

# Homography with RANSAC
if len(good) >= 4:
   src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
   dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
   H, mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
   print('Inliers:', int(mask.sum()))
   vis = cv2.drawMatches(img1, kp1, img2, kp2, good, None, matchesMask=mask.ravel().tolist())
   cv2.imwrite('sift_matches.jpg', vis)

vis_kp = cv2.drawKeypoints(img1, kp1, None,
flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
```

Analysis questions:

- What are the four main stages of SIFT (scale-space extrema, keypoint localization, orientation assignment, descriptor)?
- Why is the ratio test needed, and what happens if the ratio is 0.5 or 0.9?

- How does contrastThreshold change the number of keypoints?
- Test the same image rotated by 45 degrees and scaled by 0.5. How many matches survive?

8: ORB (Oriented FAST and Rotated BRIEF)

Objective: Use a fast binary descriptor and understand how it differs from SIFT.

Tasks:

- Create an ORB detector with cv2.ORB_create() and extract keypoints and 32-byte binary descriptors.
- Match using brute force with Hamming distance (cv2.NORM_HAMMING) and cross-check or the ratio test.
- Vary nfeatures, scaleFactor and nlevels.
- Use ORB matching for image stitching or simple object tracking in a short video.

Sample code:

```
orb = cv2.ORB_create(nfeatures=1000, scaleFactor=1.2, nlevels=8)
kp1, des1 = orb.detectAndCompute(img1, None)
kp2, des2 = orb.detectAndCompute(img2, None)
print('Descriptor shape:', des1.shape, 'dtype:', des1.dtype)  # (N, 32), uint8

bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = sorted(bf.match(des1, des2), key=lambda m: m.distance)
print('Matches:', len(matches))

vis = cv2.drawMatches(img1, kp1, img2, kp2, matches[:50], None,
                      flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
cv2.imwrite('orb_matches.jpg', vis)
```

Analysis questions:

- Why use Hamming distance for ORB but Euclidean (L2) distance for SIFT?
- How does ORB add rotation invariance to BRIEF, and what does it do about scale?
- What are the memory and speed benefits of binary descriptors?

9: SIFT vs ORB Comparison

Objective: Compare detectors and descriptors quantitatively.

Tasks:

- Create transformed copies of one image: rotations (15, 45, 90 degrees), scales (0.5, 0.75, 1.5), Gaussian blur, noise, and brightness change.
- For each transformation, run SIFT and ORB, then record keypoint count, extraction time, number of good matches, and RANSAC inlier ratio.
- Tabulate and plot the results.

Sample code:

```
import time

def evaluate(detector, norm, img, transformed, ratio=0.75):
   t0 = time.time()
   k1, d1 = detector.detectAndCompute(img, None)
   k2, d2 = detector.detectAndCompute(transformed, None)
   t = time.time() - t0
   bf = cv2.BFMatcher(norm)
   knn = bf.knnMatch(d1, d2, k=2)
   good = [p[0] for p in knn if len(p) == 2 and p[0].distance < ratio * p[1].distance]
   return len(k1), len(good), t

h, w = gray.shape
M = cv2.getRotationMatrix2D((w / 2, h / 2), 45, 1.0)
rot = cv2.warpAffine(gray, M, (w, h))
print('SIFT:', evaluate(cv2.SIFT_create(), cv2.NORM_L2, gray, rot))
print('ORB :', evaluate(cv2.ORB_create(1000), cv2.NORM_HAMMING, gray, rot))
```

Analysis questions:

- Which method is more robust to scale change? To blur? To rotation?
- When would you choose ORB over SIFT (real-time, embedded devices) and when SIFT over ORB (accuracy)?

10: HOG (Histogram of Oriented Gradients)

Objective: Use a dense gradient descriptor for object recognition.

Tasks:

- Extract HOG features and visualize them.
- Vary cell size, block size and number of orientations, and record the descriptor length.
- Train an SVM on HOG features for digit or pedestrian classification.
- Run OpenCV's built-in HOG people detector on sample images.

Sample code:

```
from skimage.feature import hog
from sklearn.svm import SVC
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

feat, hog_img = hog(gray, orientations=9, pixels_per_cell=(8, 8), cells_per_block=(2, 2),
                    visualize=True, block_norm='L2-Hys')
print('HOG length:', feat.shape)

digits = load_digits()
X = np.array([hog(im, orientations=9, pixels_per_cell=(4, 4), cells_per_block=(2, 2))
              for im in digits.images])
Xtr, Xte, ytr, yte = train_test_split(X, digits.target, test_size=0.3, random_state=0)
print('SVM accuracy:', SVC(kernel='rbf').fit(Xtr, ytr).score(Xte, yte))
```

```
hog_cv = cv2.HOGDescriptor()
hog_cv.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())
rects, _ = hog_cv.detectMultiScale(img, winStride=(8, 8), padding=(8, 8), scale=1.05)
```

Analysis questions:

- How does cell size trade off detail against feature dimension?
- Why is block normalization important